1. (2 Punkte) Was erscheint auf der Konsole?

```
def rek(n):
    if n <= 1: return 1
    return 2*rek(n-1)+4

print(rek(4))
```

Lösung: 36

2. (3 Punkte) Was erscheint auf der Konsole?

```
def rek(s):
    if len(s) == 0: return ''
    return '('+s[0]+')'+rek(s[1:])

print(rek('hal'))
```

Lösung: (h)(a)(l)

3. (2 Punkte) Was erscheint auf der Konsole?

```
def rek(s):
    if len(s) == 1: return s
    return '?'+rek(s[1:])+'/'

print(rek('hello'))
```

Lösung: ????o////

4. (3 Punkte) Was erscheint auf der Konsole?

```
def rek(s):
    if len(s) <= 2: return s
    return s[0]+s[-1]+'*'+rek(s[1:-1])

print(rek('abcdef'))
```

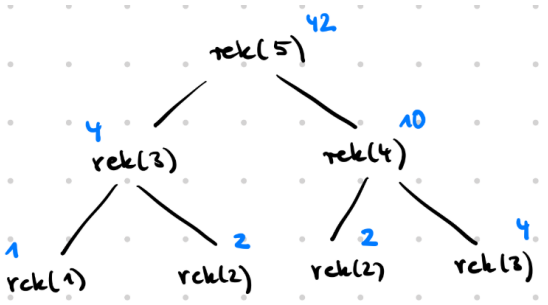
Lösung: af*be*cd

5. (3 Punkte) Gib einen geeigneten Rekursionsbaum zur Ermittlung der Zwischenergebnisse an. Was erscheint auf der Konsole?

```
def rek(n):
    if n <= 2: return n
    return 2+rek(n-2)*rek(n-1)

print(rek(5))
```

Lösung:



Die Ausgabe ist 42.

6. (3 Punkte) Gegeben ist die folgende Funktion:

```
def rek(n):
    if n == 0: return 0
    return n + rek(n-2)
```

(a) (1 Punkt) Was liefert `rek(6)`?

(b) (1 Punkt) Was passiert beim Aufruf `rek(5)`? Begründe.

(c) (1 Punkt) Ändere den Basisfall so, dass auch `rek(5)` ein Ergebnis liefert. Welches?

Lösung:

a) $6 + 4 + 2 + 0 = 12$

b) Die Aufrufe sind `rek(5)`, `rek(3)`, `rek(1)`, `rek(-1)`, ... Der Basisfall `n == 0` wird nie erreicht. Nach zu vielen geschachtelten Aufrufen (normalerweise 1000) bricht Python mit einem `RecursionError` ab.

c) `if n <= 0: return 0`. Dann ist `rek(5)` $= 5 + 3 + 1 + 0 = 9$.

7. (3 Punkte) Gib die Abfolge der rekursiven Aufrufe an. Was erscheint auf der Konsole?

```
def rek(a,b):
    if b == 0: return a+1
    return rek(2*b,a%b)

print(rek(15,20))
```

Lösung:

```
rek(15,20)
rek(40,15)
rek(30,10)
rek(20,0)
Die Ausgabe ist 21.
```

8. (2 Punkte) Was erscheint auf der Konsole?

```
def rek(a,b):
    if b == 0: return a+1
    return rek(2*b+3,a//b)

print(rek(20,10))
```

Lösung: 26

9. (3 Punkte) Gegeben sind zwei Versionen des euklidschen Algorithmus.

```
def euklid_klassisch(a,b):
    if a == b: return a
    if b > a:
        return euklid_klassisch(a,b-a)
    else:
        return euklid_klassisch(a-b,b)

def euklid_modern(a,b):
    if b == 0: return a
    return euklid_modern(b, a % b)
```

Gib für beide Versionen die Abfolge der rekursiven Aufrufe beim Aufruf mit `(91,35)` und das Ergebnis an.

Lösung:

```
euklid_klassisch(91,35)          euklid_modern(91,35)
euklid_klassisch(56,35)          euklid_modern(35,21)
euklid_klassisch(21,35)          euklid_modern(21,14)
euklid_klassisch(21,14)          euklid_modern(14,7)
euklid_klassisch(7,14)           euklid_modern(7,0)
euklid_klassisch(7,7)
Ergebnis: 7                      Ergebnis: 7
```

10. (3 Punkte) Die rekursive Funktion `sternedoppelt` gibt einen String zurück, der jedes Zeichen des übergebenen Strings durch '**' ersetzt.

Beispiel:

```
>>> sternedoppelt('lol')
'******'
```

Lösung:

```
def sternedoppelt(s):

    if len(s) == 0:
        return ''
    return '**' + sternedoppelt(s[1:])
```

11. (3 Punkte) Die rekursive Funktion `tausche` vertauscht je zwei Zeichen eines Strings. Falls die Anzahl der Zeichen ungerade ist, bleibt das letzte Zeichen an seinem Platz. Implementiere diese Funktion.

```
>>> tausche('abcde')
'badce'
>>> tausche('a')
'a'
```

Lösung:

```
def tausche(s):
    if len(s) <= 1: return s
    return s[1]+s[0]+tausche(s[2:])
```

12. (3 Punkte) Die rekursive Funktion `sterneDazwischen` fügt zwischen jedem Zeichen des übergebenen Strings das Zeichen '*' ein. Strings, die aus weniger als 2 Zeichen bestehen, bleiben unverändert. Implementiere diese Funktion.

```
'''
>>> sterneDazwischen('lol')
'l*o*l'

>>> sterneDazwischen('a')
'a'
'''
```

Lösung:

```
def sterneDazwischen(s):
    if len(s) <=1 :
        return s
    return s[0]+'*'+sterneDazwischen(s[1:])
```

13. (4 Punkte) (a) (2 Punkte) Schreibe eine rekursive Funktion `anzahl_gerade(a)`, die zurückgibt, wie viele gerade Zahlen in der Liste `a` stehen. `anzahl_gerade([4, 7, 2, 9, 6])` liefert `3`.

    (b) (2 Punkte) Schreibe eine rekursive Funktion `maximum(a)`, die die größte Zahl einer nicht leeren Liste zurückgibt, ohne `max` zu verwenden. `maximum([4, 7, 2, 9, 6])` liefert `9`.

Lösung:

```
# a)
def anzahl_gerade(a):
    if len(a) == 0: return 0
    if a[0] % 2 == 0:
        return 1 + anzahl_gerade(a[1:])
    return anzahl_gerade(a[1:])

# b)
def maximum(a):
    if len(a) == 1: return a[0]
    m = maximum(a[1:])              # Maximum des Rests
    if a[0] > m: return a[0]
    return m
```

14. (3 Punkte) (a) (2 Punkte) Schreibe eine rekursive Funktion `quersumme(n)`, die die Quersumme einer natürlichen Zahl zurückgibt. Tipp: `n % 10` ist die letzte Ziffer, `n // 10` die Zahl ohne die letzte Ziffer.

    (b) (1 Punkt) Gib die Abfolge der rekursiven Aufrufe bei `quersumme(4711)` an.

Lösung:

```
# a)
def quersumme(n):
    if n == 0: return 0
    return n % 10 + quersumme(n // 10)
```

b) `quersumme(4711)`, `quersumme(471)`, `quersumme(47)`, `quersumme(4)`, `quersumme(0)`, Ergebnis $1+1+7+4+0=13$

15. (3 Punkte) Gegeben ist die Funktion für die binäre Suche und eine sortierte Liste:

```
def binaereSuche(a,x,L=0,R=None):
    if R is None: R = len(a)-1
    if L > R: return -1
    mid = L + (R - L) // 2
    if a[mid] == x: return mid
    if x < a[mid]:
        return binaereSuche(a,x,L,mid-1)
    else:
        return binaereSuche(a,x,mid+1,R)

a = [3, 8, 15, 21, 30, 42, 57, 64, 71]
```

Gib für die Aufrufe `binaereSuche(a,64)` und `binaereSuche(a,10)` jeweils die Werte von `L` und `R` bei allen Aufrufen und den Rückgabewert an.

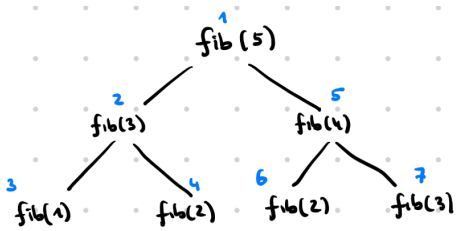
Lösung:

```
binaereSuche(a,64):  L=0 R=8,  L=5 R=8,  L=7 R=8          Rückgabe 7
binaereSuche(a,10):  L=0 R=8,  L=0 R=3,  L=2 R=3,  L=2 R=1  Rückgabe -1
```

16. (2 Punkte) Die rekursive Funktion fib wird mit fib(5) aufgerufen. Wie heißt der 6. und 7. Aufruf von fib?

```
def fib(n):
    if n <= 2: return 1
    return fib(n-2) + fib(n-1)
```

Lösung:



6. fib(2)
7. fib(3)

17. (3 Punkte) (a) (1 Punkt) Wie oft wird die Funktion `fib` insgesamt aufgerufen, wenn man `fib(6)` berechnet?

```
def fib(n):
    if n <= 2: return 1
    return fib(n-2) + fib(n-1)
```

(b) (2 Punkte) Jetzt wird die Version mit Memoization verwendet. Gib alle Aufrufe von `fib` beim Aufruf `fib(6)` der Reihe nach an und markiere die, deren Ergebnis aus `memo` kommt.

```
memo = {}

def fib(n):
    if n in memo: return memo[n]
    if n <= 2:
        res = 1
    else:
        res = fib(n-2) + fib(n-1)
    memo[n] = res
    return res
```

Lösung: a) 15 Aufrufe (1 für `fib(6)`, 5 für `fib(4)`, 9 für `fib(5)`)

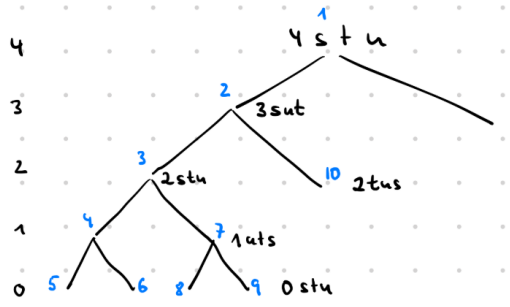
b)

```
fib(6), fib(4), fib(2), fib(3), fib(1), fib(2) memo,
fib(5), fib(3) memo, fib(4) memo
```

18. (2 Punkte) Die rekursive Funktion hanoi wird mit hanoi(4,'s','t','u') aufgerufen. Wie heißen der 9. und 10. Aufruf von hanoi?

```
def hanoi(n, start, ziel, zwischen):
    if n == 0: return
    hanoi(n-1,start,zwischen,ziel)
    print("Scheibe",n,"von",start,"nach",ziel)
    hanoi(n-1,zwischen,ziel,start)
```

Lösung:



9. hanoi(0,'s','t','u')
10. hanoi(2,'t','u','s')